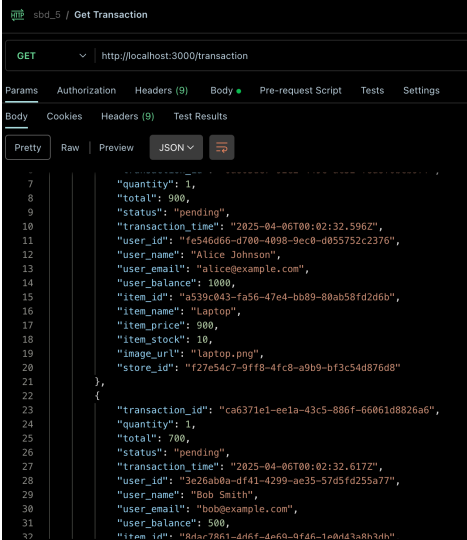
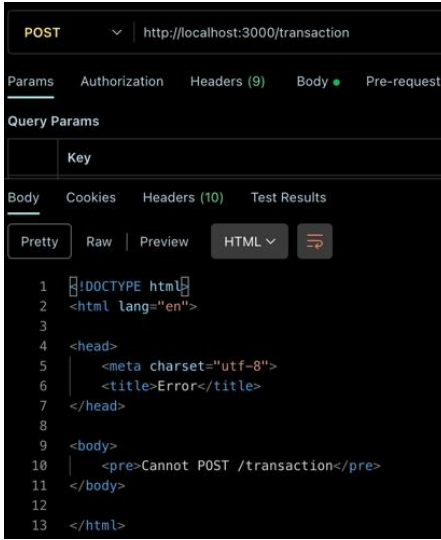


PRAKTIKUM SISTEM BASIS DATA

Nama	Bonifasius Raditya Pandu Hendrianto	No. Modul	6
NPM	2306242350	Tipe	TUTAM

1. Dokumentasi Tutam

Method	Endpoint	Success	Failed
GET	transaction/		

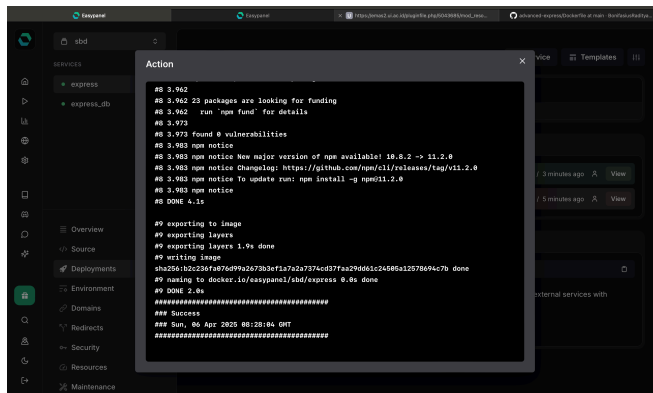
2. Implementasi Scalability, Security, Error-Handling

Tipe	Implementasi	Deskripsi
Scalability	<pre> if(!emailRegex.test(req.query.email)){ return baseResponse(res, false, 500, "Invalid Email", null); } if(!passwordRegex.test(req.query.password)){ return baseResponse(res, false, 500, "Invalid Password", null); } </pre>	Untuk memastikan password dan email yang dimasukan memiliki ketentuan sehingga mempermudah dalam pengolahan data

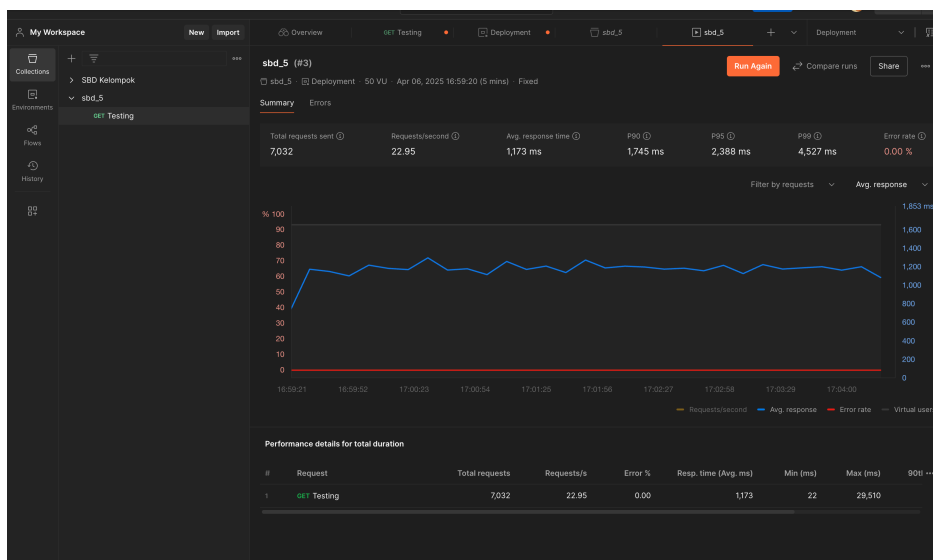
Scalability	<pre>const user = await userRepository.updateUser({ id: req.body.id, email: req.body.email, password: hashedPassword, name: req.body.name });</pre>	Memisahkan logika database dari controller sehingga pengembangan selanjutnya akan lebih mudah
Scalability	<pre>const uploadResponse = await cloudinary.uploader.upload(`data:image/jpeg;base64,\${fileStr}`); const item = { name: req.body.name, price: req.body.price, store_id: req.body.store_id, image_url: uploadResponse.url, stock: req.body.stock, };</pre>	Menggunakan cloudinary untuk penyimpanan image sehingga mengurangi beban memori database utama
Security	<pre>try { const hashedPassword = await bcrypt.hash(req.query.password, saltRounds);</pre>	Melakukan bcrypt pada password sehingga unsur password diacak untuk melindungi password dari hacker
Security	<pre>const match = await bcrypt.compare(req.query.password, user.password);</pre>	Memastikan hanya password yang sesuai dan valid yang bisa login user
Security	<pre>const uploadResponse = await cloudinary.uploader.upload(`data:image/jpeg;base64,\${fileStr}`);</pre>	Melakukan unggahan dengan format aman sehingga mengurangi risiko serangan file upload dengan memanfaatkan pihak ketiga

Error Handling	<pre>const result = await itemRepository.createItem(item); baseResponse(res, true, 201, "Item created", result); } catch (error) { console.error("Item controller error", error); baseResponse(res, false, 500, "Store Not Found", error); }</pre>	Melakukan error handling ketika item gagal dimasukan kedalam store. Kalau berhasil, output 'Item Created'
Error Handling	<pre>if(balance < total){ throw new Error("Uang Gak Cukup"); } if(stock < quantity){ throw new Error("Stock Gak Cukup"); }</pre>	Memastikan sebelum melakukan transaksi, saldo mencukupi dan stock ready sesuai permintaan pelanggan
Error Handling	<pre>if (!req.query.id !req.query.balance) { return baseResponse(res, false, 400, "Missing id or balance", null); }</pre>	Memastikan bahwa rekening masuk kedalam database sebelum melakukan transaksi
Error Handling	<pre>if (transactions.rows.length === 0) { throw new Error("No transactions found"); }</pre>	Memastikan saat ingin GET dan nilainya tidak ada, maka pencarian tidak ditemukan

Deploy dockerfile dan berhasil



Hasil postman



Stress test pada postman tersebut berhasil dikarenakan total request yang diberikan adalah 7022 sehingga server ini mampu menangani permintaan dengan volume yang tinggi. Responsiveness juga stabil di angka 22.16 rps secara konsisten. Avg. response time berada di angka 1.173, P95 di angka 2.38 ms (95% response selesai dibawah 2.38 ms), dan P99 di angka 4.527 ms (99% response selesai dibawah 4.5 ms) yang menandakan server sangat responsive karena rata-rata waktu respon cenderung rendah dan stabil karena tidak ada lonjakan signifikan (timeout). Error rate dari server ini berada di angka 0.00 yang artinya semua request berhasil diproses tanpa adanya error sehingga performa dinilai sangat baik. Grafik interval waktu (warna biru) juga cenderung stabil dan tidak ada bouncing atau spiking sama sekali.